

Giải Phẫu Index (Anatomy of an Index)

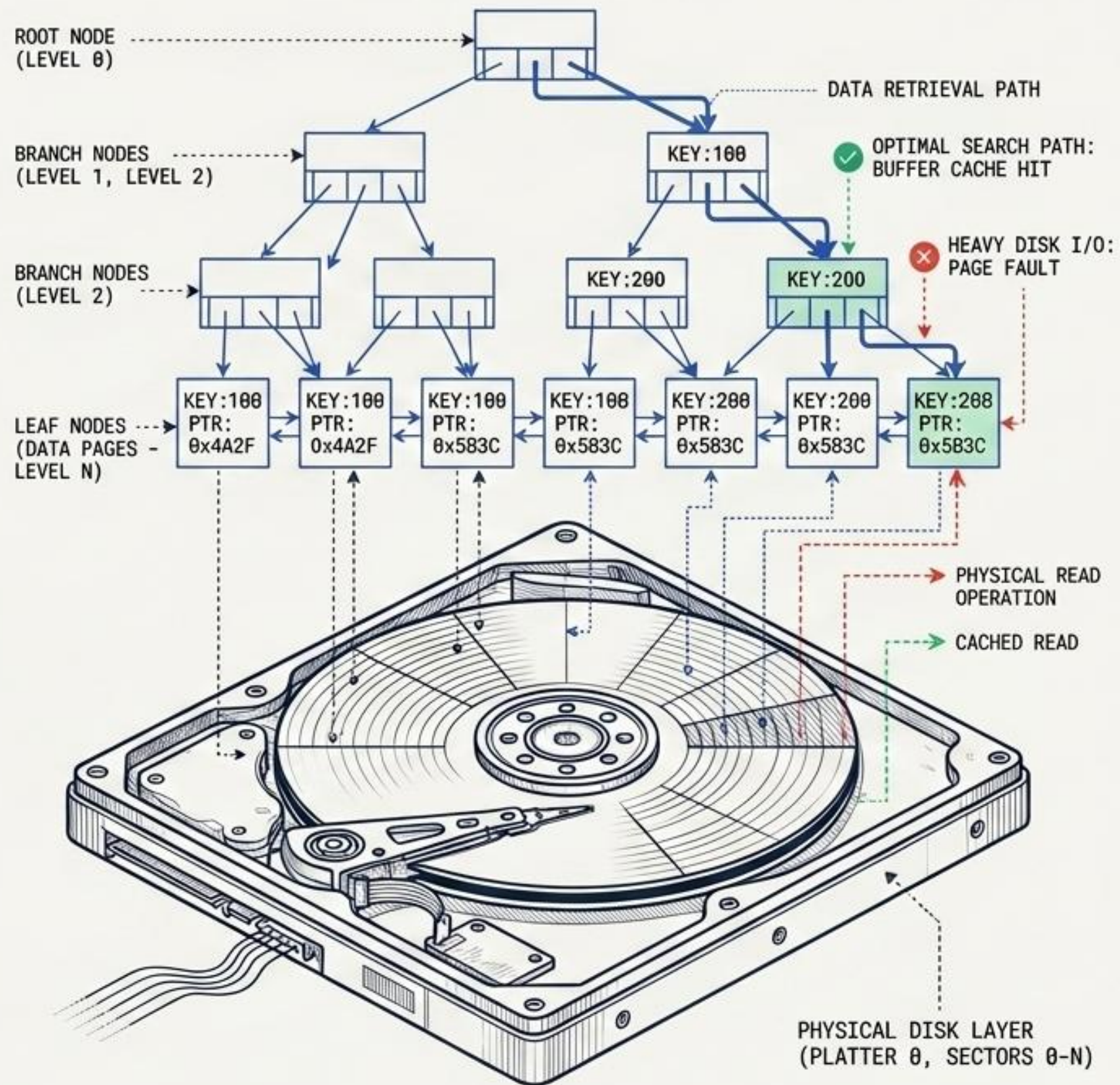
Hiểu sâu về cấu trúc cây B-Tree và cách Database tìm kiếm dữ liệu dưới tầng vật lý

1. Bản chất thực sự của một Index

2. Tầng Lá vs. Tầng Dẫn Đường

3. Cơ chế duyệt cây (Tree Traversal)

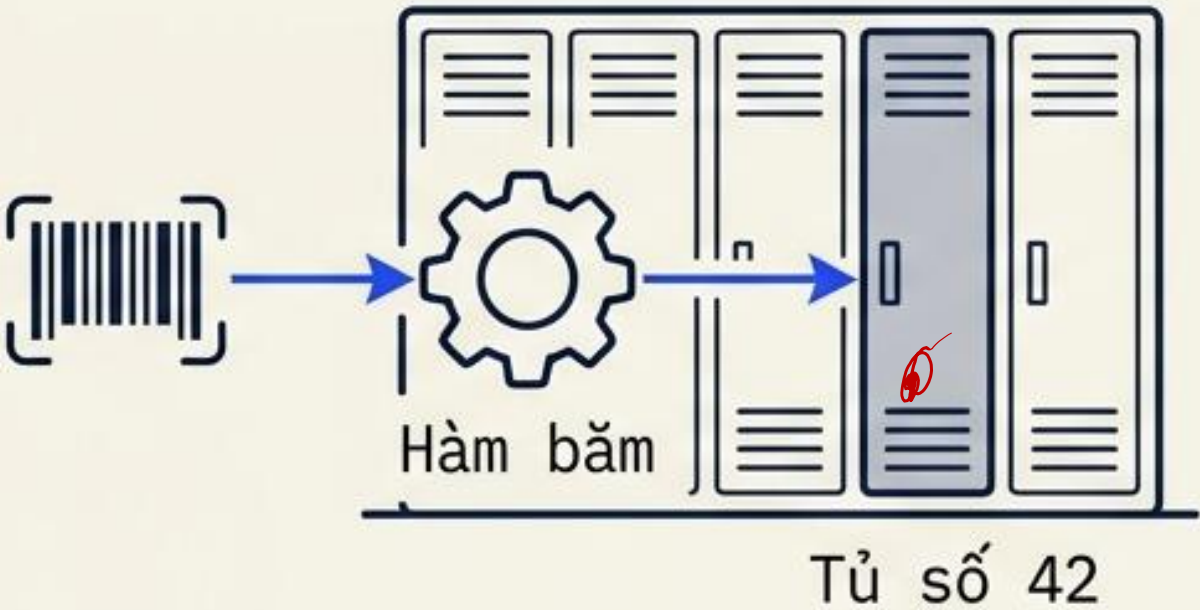
4. Độ sâu và tốc độ hệ thống



Phản biện 1: Hash Table (Bảng băm) rất nhanh, nhưng...

	Diagnostic Matrix	
1.	Tốc độ:	■ Tốt
2.	Truy vấn khoảng:	■ Thất bại
3.	Chi phí Ghi đĩa:	■ Trung bình

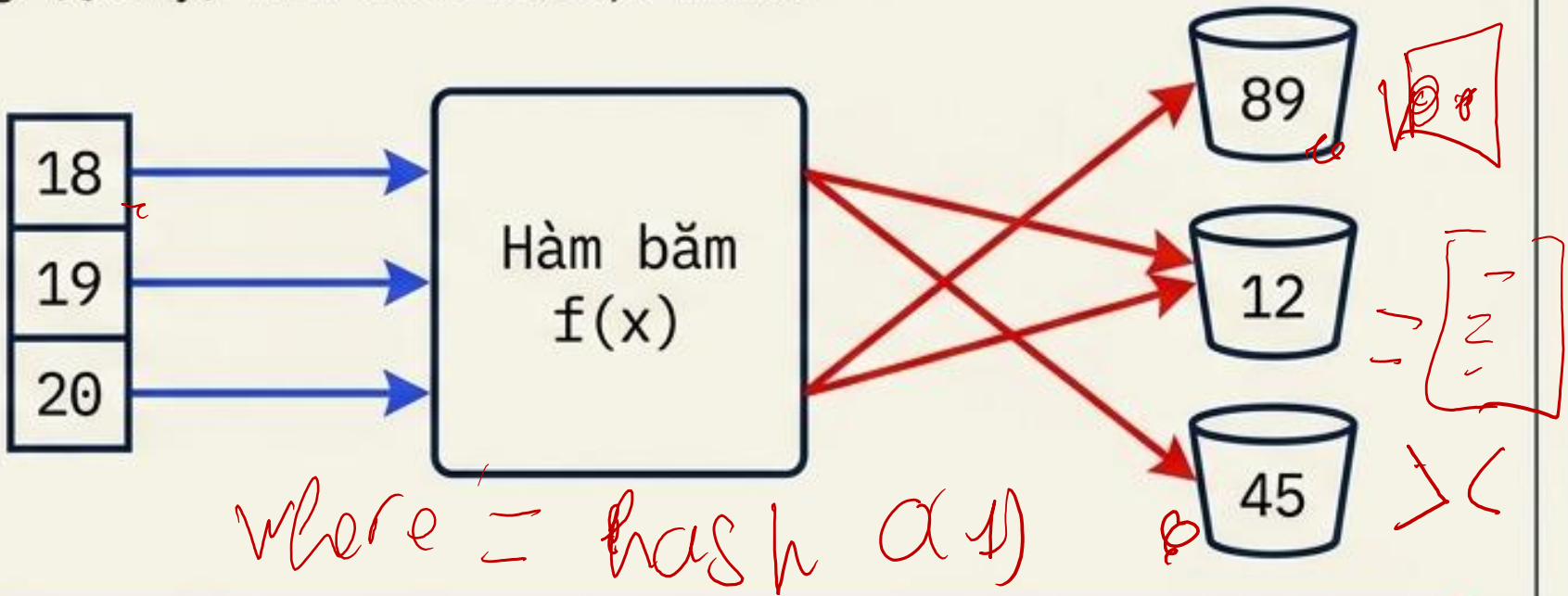
Hash Table là gì?



Ưu điểm vượt trội: Tìm kiếm chính xác cực nhanh với độ phức tạp thuật toán tối ưu nhất: $O(1)$ (chỉ mất đúng 1 bước so sánh).

Fatal Flaw

Chí mạng: Hash Table (Bảng băm băm, khoảng với truy vấn khoảng (Query) như theo khoảng này..



Chí mạng (Fatal Flaw): Thất bại hoàn toàn với truy vấn khoảng (Range Query) như WHERE age > 18 và không hỗ trợ ORDER BY. Dữ liệu bị xáo trộn ngẫu nhiên.

Phản biện 2: Tìm kiếm nhị phân (Binary Search) thì sao?

Diagnostic Matrix	
Tốc độ:	Tốt ✓
Truy vấn khoảng:	Tốt ✓
Chi phí Ghi đĩa:	Ác mộng

Top Panel (Logic)

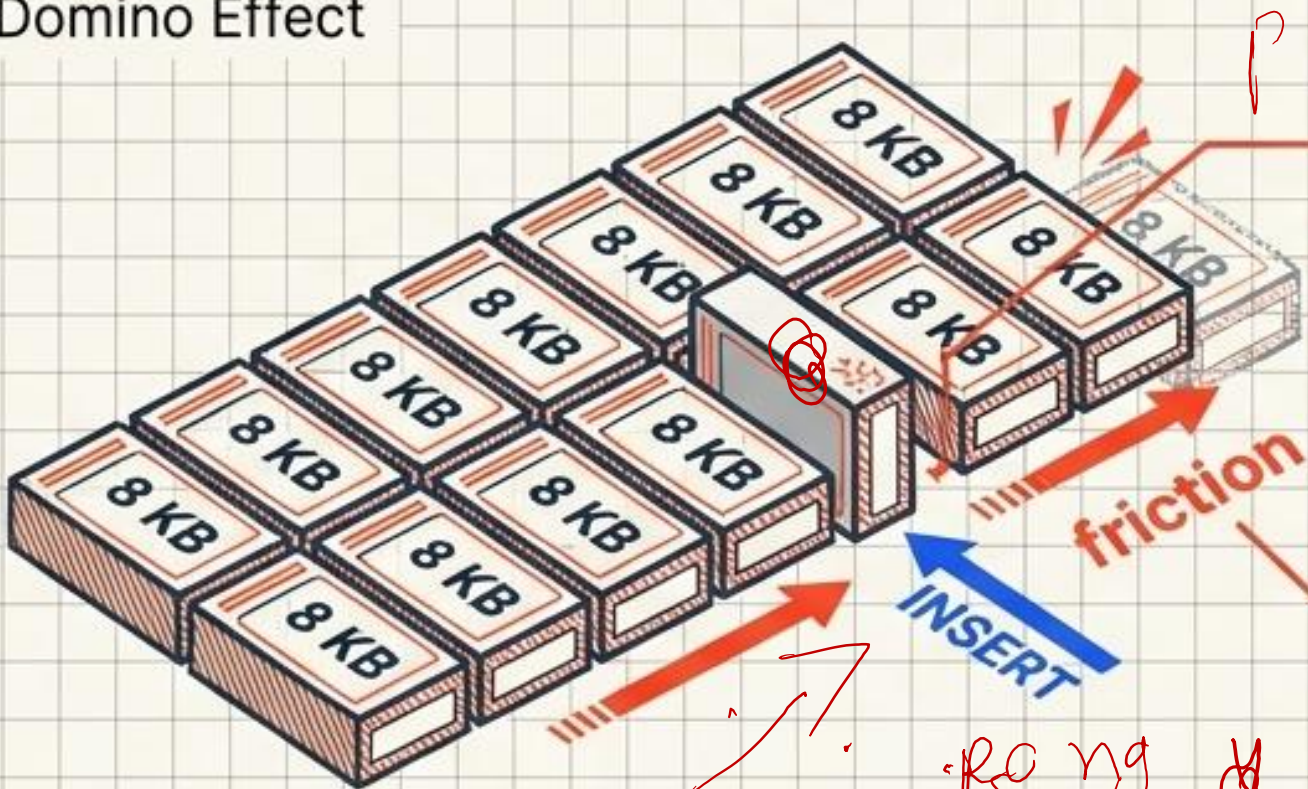
$O(\log N)$ Halving Concept



$O(\log N)$

Ưu điểm: Tìm kiếm trên một mảng đã sắp xếp sẵn với tốc độ cực nhanh: độ phức tạp $O(\log N)$.

Physical Reality - The Domino Effect



Chỉ mạng 1 (Disk I/O): Dữ liệu lưu theo Page (8 KB / 16 KB). Nhảy ngẫu nhiên ép Database nạp liên tục các Page mới từ ổ đĩa lên RAM.

ra ở đây → RAM

Chỉ mạng 2 (Write Cost): Chèn dữ liệu (INSERT/DELETE) tạo ra cơn ác mộng dịch chuyển hàng triệu dòng trên ổ đĩa vật lý.

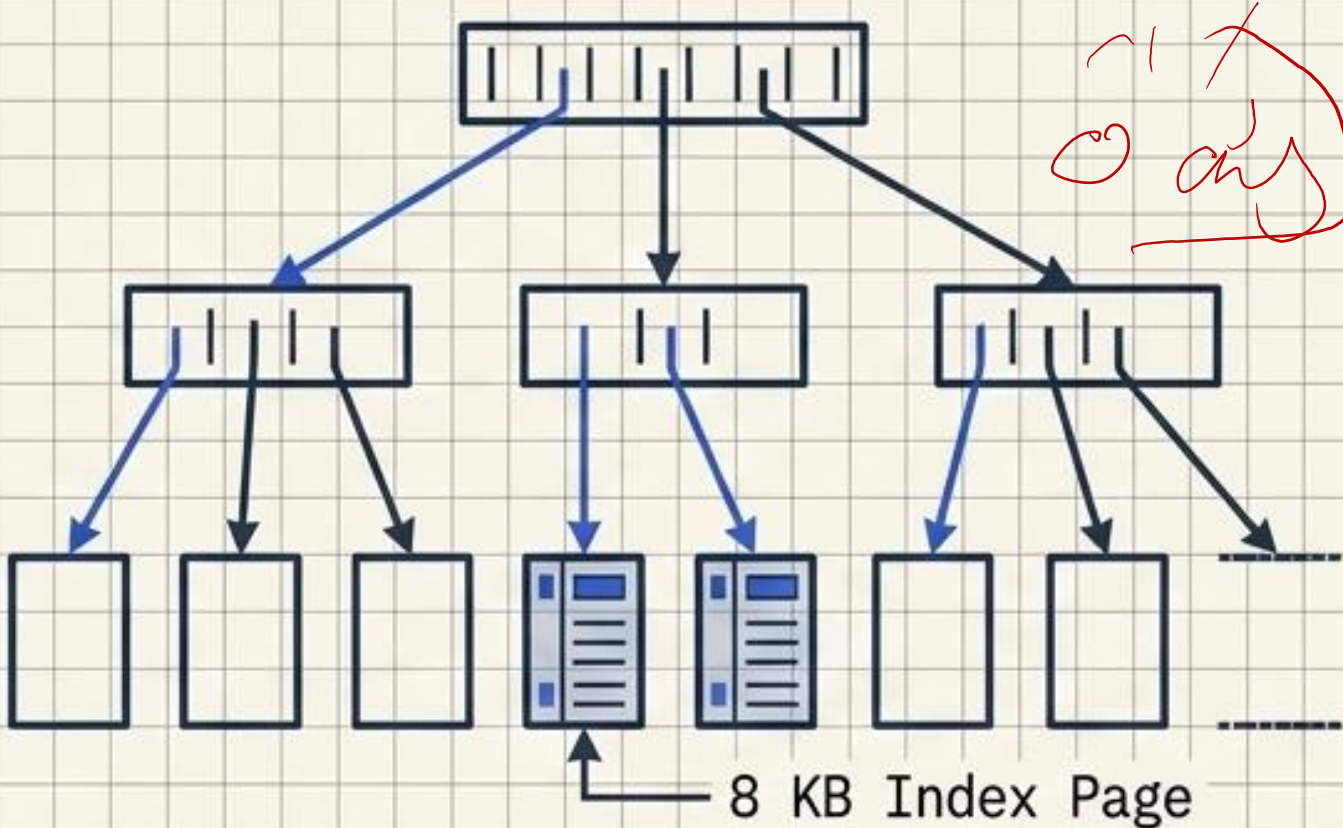
reng & s ch xeng

Làm tương kinh điển về Index

Diagnostic Matrix	
Tốc độ:	Tốt
Truy vấn khoảng:	Tốt
Chi phí Ghi đĩa:	Tối ưu



VS

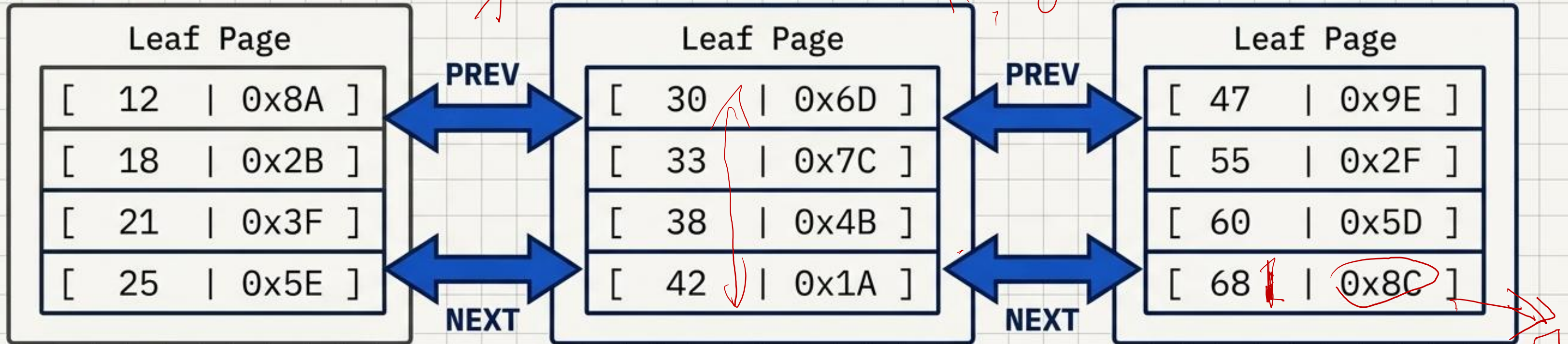


Làm tương: Chỉ là cuốn danh bạ điện thoại (mò mẫm theo vần).

Thực tế vật lý: Cấu trúc sắp xếp (Sorted Structure) dạng cây cân bằng (Balanced Tree).

Kết hợp hoàn hảo giữa tìm kiếm có định hướng và lưu trữ phân mảnh theo Page.

Tầng Lá (Leaf Nodes): Điểm cuối của hành trình



Tính sắp xếp (Sorted):

Dữ liệu luôn tuân thủ thứ tự logic nghiêm ngặt (VD: 1 -> 100).

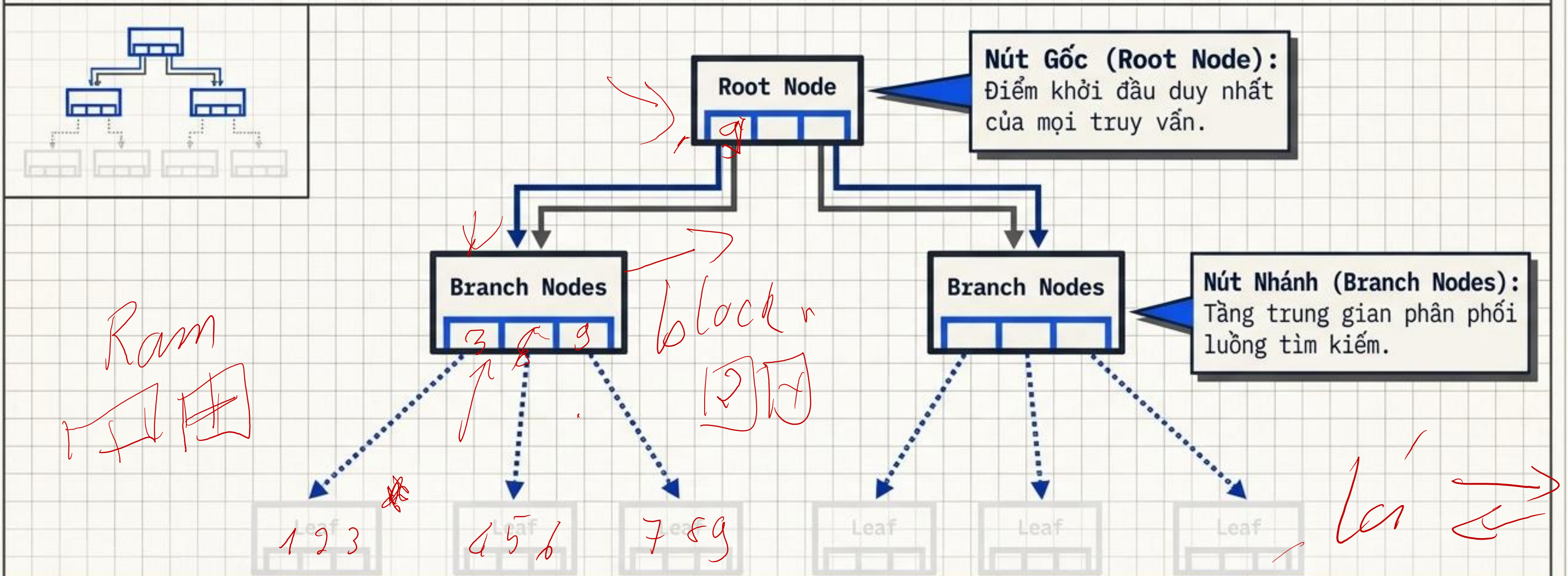
Liên kết đôi (Doubly Linked List):

Con trỏ PREV và NEXT cho phép quét xuôi/ngược cực nhanh mà không cần vòng lại nút gốc.

Nội dung cột lỗi:

Mỗi phần tử chứa Index Key (Giá trị cột) và ROWID (Địa chỉ vật lý đích xác trên Data Page).

Nút Gốc (Root) & Nút Nhánh (Branch): Bản đồ chỉ đường



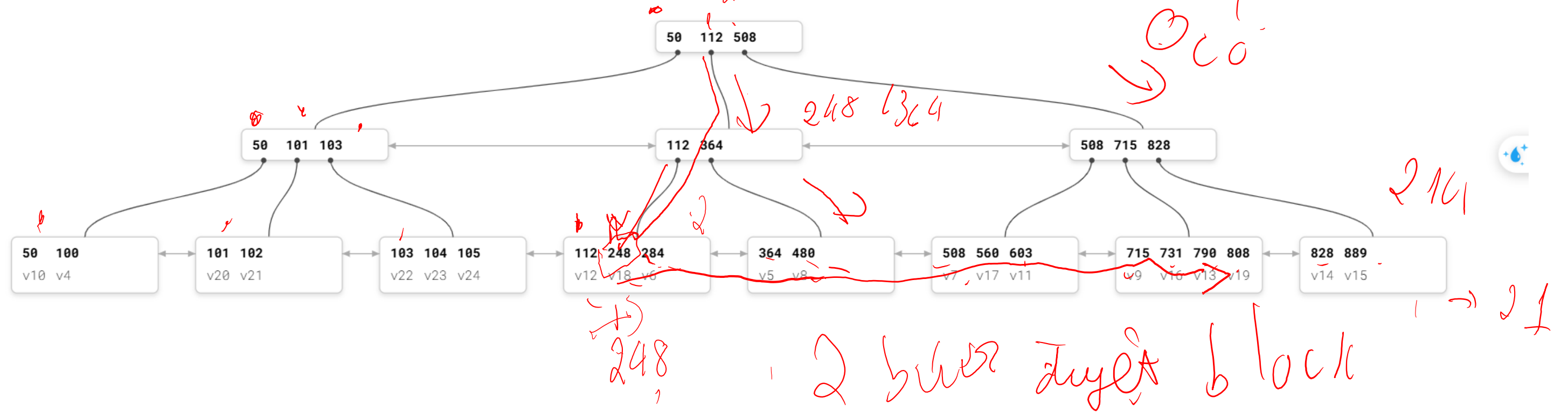
Cảnh báo / Insight: KHÔNG chứa dữ liệu thực tế (No ROWIDs). Chúng chỉ chứa Khóa phân vùng và Con trỏ hướng xuống tầng thấp hơn.

3 bước

tim kiếm
 $248 < 508$

$248 > 100 < 508$

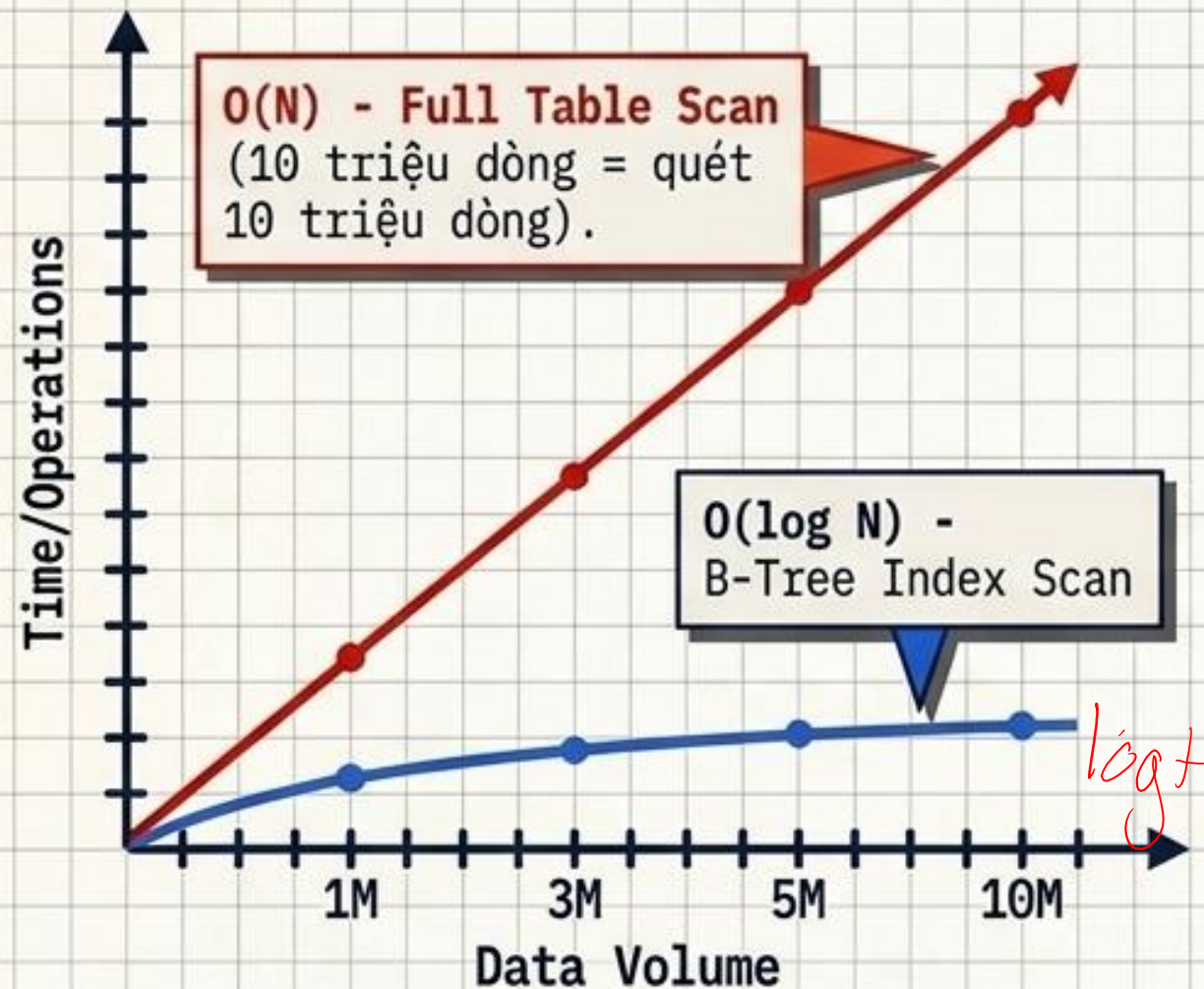
Ồ có



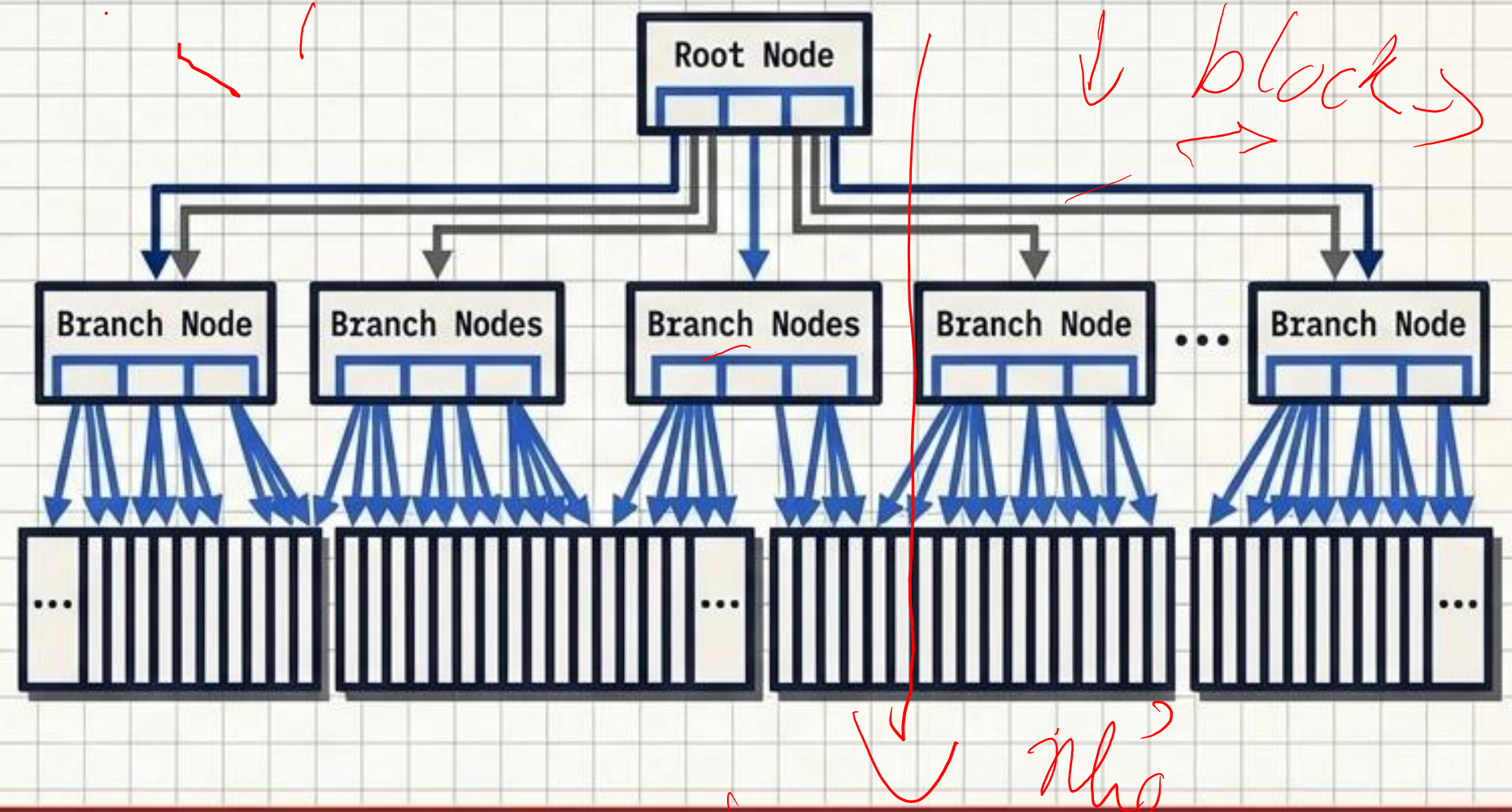
where $248 \rightarrow 790$
 $IP > 248$ and $IP < 790$

Sức mạnh của độ phức tạp $O(\log N)$

Performance Graph



The Flattened Tree Structure



Tư duy Architect:

- **1 Triệu Dòng:** Cây sâu 3 tầng -> Mất đúng 3 lần đọc Page (3 Disk I/Os).
- **10 Triệu Dòng (Gấp 10 lần):** Cây sâu 4 tầng -> Chỉ tốn thêm đúng 1 Disk I/O. Tốc độ gần như không đổi!

Handwritten notes in red:
 3 lần 3 I/O → 8 lần 4
 10 lần → 4 → 2 lần + 2 lần

Chữ "B" trong B-Tree thực chất là gì?

Sự thật: "B" là **Balanced** (Cân bằng), KHÔNG PHẢI **Binary** (Nhị phân).
Khoảng cách từ Root đến mọi Leaf luôn bằng nhau 100%.

Trước (Before)

Đầy (Full) do INSERT

Leaf Node

[10 22 35 48]
[55 61 73 89]
[58 55 73 99]
[99 105 112]

balance gốc
↓
thông
lại đơn

Page Split
(Tách trang)

Sau (After)

Branch Node

[61]

Leaf Node

[10 22 35]
[48 55]

Leaf Node

[61 73 89
[99 105 112]

Cân bằng Hoàn hảo
(Perfect Balance)

Hiệu năng ổn định

Chuyện gì xảy ra sau khi tìm thấy ROWID?

Logical Index (trong số logical) ROWID

SELECT *
(Lấy TOÀN BỘ các cột, không chỉ user_id)

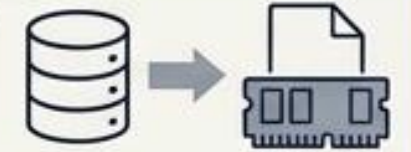
Leaf Node

Key: [42]

ROWID

Table Access by Index ROWID

Database phải cầm ROWID, bay xuống ổ đĩa, bốc đúng Data Page đó lên RAM để lấy các cột còn lại (email, name...).



! **Tư duy cốt lõi:** Nếu truy vấn quá nhiều dòng, việc 'bay qua bay lại' liên tục xuống Data Page sẽ tạo ra thảm họa Disk I/O.

Data Pages

Bài sau: Khi nào Database thà dùng Full Table Scan còn hơn dùng Index?